
Mise à niveau en Informatique

Programmation Java – TP

Ce document est le guide à utiliser pour réaliser les séances de TP de programmation en Java, dans le cadre de la mise à niveau en Informatique. Ce document comprend les éléments suivants :

- quelques rappels de base sur l'exécution de programmes Java
- quelques modalités pratiques sur le déroulement des séances de TP
- une collection d'exercices de programmation

Lisez attentivement et comprenez les premières parties explicatives avant de commencer à programmer.

1 Exécution de programmes Java

Un programme Java est exécutable sur une machine virtuelle, la Java Virtual Machine (JVM). La JVM embarque tout le nécessaire pour exécuter un programme Java : environnement d'exécution, gestion de la mémoire, etc. Le principal avantage d'une machine virtuelle est la portabilité : une fois un programme compilé en Bytecode Java, le langage exécuté par la JVM, n'importe quelle JVM compatible peut exécuter ce programme, et en particulier, indépendamment du système d'exploitation sous-jacent, et de la machine physique utilisée. L'adage Java est *"Write once, run anywhere"*.

Il est important de connaître ces mécanismes de base, qui sont mis en oeuvre par les environnements de programmation intégrés. Pour découvrir, mieux comprendre et maîtriser ces mécanismes de base, vous devrez donc utiliser un environnement de programmation minimal durant la première séance de TP.

2 Modalités pratiques

Pour réaliser les exercices de programmation en Java, vous utiliserez donc :

- 1ère séance encadrée : un simple éditeur de texte (JEdit, TextEdit, Atom...) pour définir vos programmes, et l'interface en ligne de commande (dans un terminal) pour compiler et exécuter les programmes compilés.
- Les séances suivantes : un environnement de développement intégré, Eclipse, qui intègre un éditeur, le processus de compilation, ainsi qu'une console intégrée pour les entrées/sorties. Eclipse fournit aussi plusieurs autres fonctionnalités avancées (debugger, suivi de tâches, test unitaires...).

2.1 Programmer, compiler, exécuter

Durant la première séance encadrée, vous utiliserez des outils très basiques pour programmer. Le but est de vous initier aux mécanismes de base en oeuvre dans l'activité de programmation avec les langages compilés, comme Java.

Pour rappel,

- Un programme Java est écrit dans un (ou plusieurs) fichier(s) source `.java`,
- Le programme peut utiliser toute la bibliothèque standard Java. Éventuellement, le programme peut également utiliser un ensemble de bibliothèques ad-hoc.
- Le programme source et les bibliothèques ad-hoc éventuelles doivent être **compilés** avec un compilateur Java. Le processus de compilation produit des fichiers bytecode Java, d'extension `.class`. Le compilateur Java que vous utiliserez dans l'interface en ligne de commande est l'exécutable `javac`, pour *Java compiler*.
- Quand la compilation a entièrement réussi, sans erreur, le programme peut ensuite être exécuté au travers de la machine virtuelle Java. Pour s'exécuter correctement, toutes les bibliothèques utilisées doivent aussi être accessibles à la machine virtuelle Java, via son *class-path*. Pour cela, on peut spécifier un ensemble de répertoires à inclure dans le *class-path*, en plus de la bibliothèque standard. En ligne de commande, l'exécutable à utiliser pour la machine virtuelle Java est `java`, et l'option `-cp` (pour *class-path*) permet d'indiquer les répertoires à ajouter au class-path par défaut.
- Selon les configurations système des salles de TP, vous pourriez aussi avoir besoin d'utiliser l'option du class-path `-cp` dès l'étape de compilation.
- À noter : pour configurer le class-path, il est également possible d'utiliser une variable d'environnement pour votre système d'exploitation. Pour plus d'information :
<https://docs.oracle.com/javase/tutorial/essential/environment/paths.html>

Voici un exemple illustrant ce processus de développement. Supposons qu'un fichier `Tp0.java` existe dans le répertoire courant, et qu'il contienne le programme suivant :

```
public class Tp0 {  
  
    public static void main (String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

Ci-dessous, voici un exemple d'interactions que vous pourrez avoir dans l'interface en ligne de commande pour consulter le répertoire courant, compiler le programme source, puis exécuter le programme compilé.

```
$ ls                # la commande ls liste le répertoire courant  
Tp0.java           # le répertoire ne contient qu'un seul fichier source  
$ javac Tp0.java   # la commande javac compile le fichier  
                  # la compilation réussit, sans message d'erreur  
$ ls                # le fichier compilé est maintenant présent  
Tp0.class          Tp0.java
```

\$ java Tp0	<i># la commande java exécute un programme Java</i>
	<i># On exécute ici le main de la classe Tp0</i>
	<i># /\ indiquer le NOM de la classe, sans extension .class</i>
Hello World!	<i># les entrées/sorties sont réalisées dans le terminal</i>
\$	<i># fin de l'exécution du programme, la JVM rend la main</i>

2.2 Environnement de développement intégré Eclipse

Après la première séance, vous utiliserez un environnement de développement intégré, Eclipse, qui se base sur les mêmes mécanismes de base que précédemment, mais qui fournit :

- un éditeur de texte pour les programmes source Java
- un processus de compilation intégré à Eclipse. Le processus de compilation est, par défaut, déclenché à chaque sauvegarde de fichier (automatique, ou à la sauvegarde explicite par l'utilisateur). Les avertissements (warnings) et erreurs de compilation sont rapportés directement dans l'éditeur, et repérés par des triangles jaunes ou des croix rouges. Tant que le processus de compilation échoue (comporte des erreurs), vous ne pourrez pas exécuter un programme. Vous ne pourrez exécuter que la dernière version compilée avec succès du programme.
- une console pour gérer les entrées/sorties entre le programme et l'utilisateur.

Eclipse est déjà installé sur les machines des salles de TP. Il faudra vous créer un Workspace (un espace de travail pour vos TP de Njava), et créer des *projets* Eclipse Java. Demandez à vos encadrants de vous aider à utiliser Eclipse en cas de besoin (configuration, utilisation...).

3 Exercices - Ingrédients de base

Exercice 3.1 (Parallélépipède rectangle). Un parallélépipède rectangle (ou pavé droit, ou boîte rectangulaire) est un solide dont les 6 faces sont des rectangles. En s'inspirant du programme `Sphere` vu et expliqué en cours, définir un programme Java `Parallelepiped` qui calcule la surface et le volume de parallélépipèdes rectangles de diverses dimensions. Le programme devra, par exemple, afficher la surface et le volume des parallélépipèdes suivants :

- un premier de dimensions $12 \times 3 \times 5$
- et un deuxième de dimensions $4 \times 34 \times 10$

Indication : aidez-vous d'un dessin représentant les parallélépipèdes, en y indiquant le nom de chaque dimension.

Exercice 3.2 (Moyenne de 3 entiers). Définir un programme `Average` qui calcule la moyenne de 3 nombres de type `int`. Exemples :

- les trois entiers 3, 1 et 1
- les trois entiers -12 , 2 et 12

Exercice 3.3 (Moyenne de 3 entiers, paramètres du programme). Définir un programme `AverageArgs` qui calcule la moyenne de 3 nombres entiers passés en paramètres au programme sur la ligne de commande.

Indication : vous devrez pour cela utiliser dans la fonction `main` son paramètre. C'est un tableau de type `String[]` un tableau de chaînes de caractères. Pour convertir les chaînes qu'il contient en des entiers avec lesquels on peut calculer, vous pourrez utiliser la fonction Java `int Integer.parseInt(String s)`.

Exercice 3.4 (Surface de couronnes). On souhaite calculer la surface de couronnes délimitées par deux cercles concentriques de rayons r_1 et r_2 . Les rayons peuvent être quelconques : r_1 peut être supérieur, égal, ou inférieur à r_2 . Pour cela, on décide de définir deux fonctions :

- `surfaceCercle` : surface d'un cercle de rayon donné
 - `surfaceCouronne` : surface d'une couronne de rayons donnés
1. Réalisez un dessin illustrant un exemple de couronne, en y indiquant les noms des rayons des deux cercles.
 2. Définir un programme `Couronne` qui calcule et affiche la surface d'une couronne délimitée par deux cercles
 - (a) de rayons respectifs $r_1 = 2.3$ et $r_2 = 4.1$
 - (b) de rayons respectifs $r_1 = 4.1$ et $r_2 = 2.3$
 - (c) de rayons passés en paramètres au programme. *Indication : vous pourrez utiliser la fonction Java `double Double.parseDouble(String s)`*

Indication : vous pourrez utiliser la fonction `double Math.abs(double x)` pour le calcul de valeur absolue, et la constante `Math.PI` pour la valeur double de π .

Exercice 3.5 (Modularité). On désire calculer le périmètre et l'aire de cercles et de rectangles. Pour des raisons de modularité, on décide de répartir les définitions de fonctions dans plusieurs classes :

- une classe **Circle**, spécialisée dans les calculs concernant les cercles,
- une classe **Rectangle** spécialisée dans les calculs concernant les rectangles,
- une classe **Calculs** contenant la procédure principale **main**, qui utilise les deux classes précédentes, en testant leurs fonctions pour un cercle de rayon 12 et pour un rectangle de côtés 12 et 15.

Définir ces trois classes, chacune dans un **fichier différent**. Toutes leurs fonctions seront définies avec le mot-clef Java **static** : il ne s'agit pas ici de faire de la programmation objet.

4 Exercices - Instructions conditionnelles

Exercice 4.1 (Calcul de maximum). Dans un programme **Maximum**, définir et tester les fonctions suivantes :

- Une fonction **max** qui calcule le plus grand de deux entiers donnés en paramètres.
- Une fonction qui calcule le plus grand de trois entiers donnés en paramètres. Vous définirez deux versions :
 1. **max3v1** utilisera la fonction précédente **max**
 2. **max3v2** sera programmée directement, sans utiliser **max**

Exercice 4.2 (Calcul de médiane). Dans un programme **Mediane** définir une fonction qui calcule la médiane de trois entiers, c'est-à-dire l'entier qui est "encadré" par les deux autres. Exemples :

- la médiane de 8, 1 et 4 est 4 ;
- la médiane de 4, 1, 1 est 1 ;
- la médiane de 1, 4, 4 est 4.

Exercice 4.3 (Équation du second degré). Définir un programme **Equation2Degré** qui calcule puis affiche la ou les solutions d'une équation de second degré :

$$ax^2 + bx + c = 0$$

Attention : faire une analyse sérieuse. Les nombres a, b, c sont des nombres quelconques. Ils peuvent notamment être nuls.

*Conseil : définir une **fonction** auxiliaire qui rende en résultat une chaîne de caractères signifiant «en clair» le nombre et la nature des solutions.*

5 Exercices - Itérations et récursivité

Exercice 5.1 (*i*-ème chiffre). Étant donné un entier $n \geq 0$, on souhaite trouver le *i*-ème chiffre de son écriture décimale.

Les chiffres sont numérotés à partir de 0, et en partant de la droite. Si *i* est plus grand que le rang du plus fort chiffre significatif, le résultat sera 0. Exemple : pour $n = 245$,

- Le 0^e chiffre de n est 5
- Le 1^{er} chiffre de n est 4
- Le 2^e chiffre de n est 2
- À partir de $k = 3$, le k^e chiffre de n est 0

Définir une fonction `digit` réalisant ce calcul. Vous définirez deux versions de cette fonction : une version itérative et une version récursive.

Exercice 5.2 (Somme des chiffres). Définir une fonction `sumOfDigits` qui calcule la somme des chiffres de l'écriture décimale d'un entier positif ou nul. Vous définirez deux versions de cette fonction : une version itérative, et une version récursive.

Exercice 5.3 (Fibonacci). La fameuse suite de Fibonacci est définie de la manière suivante :

$$\begin{aligned} F_0 &= 1 & F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \text{ pour } n > 1 \end{aligned}$$

Définir une fonction `int fibo(int n)` qui calcule F_n , le *n*-ième terme de cette suite. On définira deux versions pour cette fonction : une version récursive et une version itérative.

6 Exercices - Itération et données entrées au clavier

Pour réaliser ces exercices, la lecture des données sera réalisée au cours de l'exécution du programme, en les entrant au clavier. On utilisera donc la classe Java `Scanner`. Pour ces exercices, il est inutile de passer par des tableaux d'entiers.

Exercice 6.1 (Moyenne). Définir un programme `Moyenne` qui lit une suite non vide de nombres entiers positifs, terminée par -1, et calcule puis affiche la moyenne de ces nombres. Exemple : Pour la suite d'entiers 4 3 0 4 -1, la moyenne est 2.75.

Exercice 6.2 (Sous-suite constante). On entre au clavier une suite non vide d'entiers positifs terminée par la marque -1. Définir un programme qui lit ces nombres et calcule la longueur de la plus longue sous-suite constante. Exemple : Pour la suite d'entiers 7 5 5 4 1 1 1 1 8 8 4 1 -1, la longueur de la plus longue sous-suite constante est 4 (suite de quatre 1).

7 Exercices - Tableaux

Exercice 7.1 (Indice du maximum). Définir une fonction `indexOfMax` qui, étant donné un tableau d'entiers, calcule un indice de l'entier maximum dans ce tableau. *Indication : attention aux pré-requis de la fonction.*

Exercice 7.2 (Élément maximum). Définir une fonction `maxTab` qui, étant donné un tableau d'entiers, calcule l'entier maximum dans ce tableau. *Indication : attention aux pré-requis de la fonction.*

Exercice 7.3 (Renforcement). Réaliser l' Exercice 6.1, en utilisant cette fois un tableau d'entiers de longueur non nulle quelconque, qui sera le paramètre de la fonction. *Indication : on évitera d'utiliser la marque -1, qui n'est plus utile dans ce cadre.*

Exercice 7.4 (Renforcement). Réaliser l' Exercice 6.2, en utilisant cette fois un tableau d'entiers de longueur non nulle quelconque, qui sera le paramètre de la fonction. *Indication : on évitera d'utiliser la marque -1, qui n'est plus utile dans ce cadre.*

Exercice 7.5 (Fusion de tableaux). Soient deux tableaux `t1` et `t2` de nombres entiers, tous deux triés par ordre croissant. Définir une fonction `fusion` qui calcule un nouveau tableau d'entiers trié, contenant tous les éléments de `t1` et `t2`.

8 Exercices - Chaînes de caractères

Pour les exercices de cette partie, la seule opération sur les chaînes de caractères que vous devez utiliser est la méthode `char charAt(int i)`. Les autres ne sont pas nécessaires.

Exercice 8.1 (Nombre d'occurrences). Définir une fonction `nbOcc` qui, étant donnés une chaîne de caractères `s` et un caractère `c` quelconque, calcule le nombre d'occurrences de `c` dans `s`, c'est-à-dire le nombre de fois où il y apparaît. Exemple : il y a 5 occurrences de 'a' dans "abracadabra", et il y a 0 occurrence de 'e' dans "Njava".

Exercice 8.2 (Palindrome). Définir une fonction `estPalindrome` qui détermine si une chaîne de caractères quelconque est un palindrome. Pour rappel, un palindrome est une chaînes de caractères où l'ordre d'apparition des caractères est le même quand elle est lue de gauche à droite, et de droite à gauche. Exemple : Les chaînes "kayak" "ABBA" et "LOL" sont des palindromes. La chaîne "baba" n'est pas un palindrome.

Exercice 8.3 (Nombre de mots). Définir un programme `CompteMots` qui détermine puis affiche le nombre de mots contenus dans une phrase terminée par un point.

Pour simplifier, on considère que le seul caractère séparateur de mots est le caractère espace. Mais attention, les espaces sont possibles avant ou après le premier mot, et plusieurs espaces consécutifs sont aussi possibles.

9 Exercices - Programmation Objet

Exercice 9.1 (Nombres complexes - Structure). Un nombre complexe peut être représenté au moyen d'une structure à deux champs, le champ `x` (partie réelle) et le champ `y` (partie imaginaire).

Définir cette représentation au moyen d'une classe structure **Complex**.

Définir un programme **CalcComplex** dans une classe séparée, qui contiendra les fonctions statiques suivantes :

- `static Complex add (Complex z1, Complex z2)` qui rend en résultat la somme de deux complexes.
- `static Complex mul(Complex z1, Complex z2)` qui rend en résultat le produit de deux complexes.
- `static String toString(Complex z)` qui rend en résultat une chaîne de caractères de la forme "`x+y.i`" dénotant le nombre complexe `z` sous la forme usuelle.

Attention, on fera en sorte de rendre des chaînes simplifiant les écritures lorsque la partie réelle et/ou imaginaire valent 0 et/ou 1. Exemple : on rendra

- "`x`" au lieu de "`x+0.i`"
- "`y.i`" au lieu de "`0+y.i`"
- "`x+i`" au lieu de "`x+1.i`"
- "`i`" au lieu de "`0+i`"
- et "`0`" au lieu de "`0+0.i`"
- une procédure principale `main` qui lit deux réels `x` et `y`, crée le nombre complexe `z` correspondant, puis calcule et affiche le nombre complexe $z.(z.(z + 1) + 1)$.

Remarque : il faudra au préalable définir le nombre complexe 1, c'est-à-dire $1 + 0.i$, pour l'utiliser dans les opérations à réaliser.

Exercice 9.2 (Nombres complexes - Classe objet). À l'exercice précédent, nous avons représenté les nombres complexes au moyen d'une structure. Ce n'est pas vraiment la bonne façon de faire. Il vaut mieux totalement abstraire les données qui représentent un nombre complexe (`x` et `y` dans cet exemple), définir une classe objet incluant les fonctions de manipulation des complexes, et ne rendre visible aux utilisateurs que ces fonctions-là.

Définir une classe **OComplex** dans cet esprit, et définir un programme **OComplexCalc** dont la procédure principale réalisera la même chose qu'à l'exercice précédent, mais en utilisant cette fois la classe **OComplex**.

Pour satisfaire les goûts divers et variés des utilisateurs potentiels, on offrira dans la classe **OComplex** deux versions de chaque fonction de calcul sur les complexes. Par exemple, pour la fonction d'addition :

1. une version fonction statique :

```
public static OComplex add(OComplex z1, OComplex z2)
// résultat : la somme de z1 et de z2
```
2. une version méthode d'objet :

```
public OComplex add(OComplex z)
// résultat : la somme de this et de z
```

Exercice 9.3 (Parc locatif - Classe objet). Une agence immobilière souhaite analyser l'ensemble de son parc immobilier en location. Pour cela, on décide de :

- représenter un appartement par son adresse (une chaîne de caractères quelconque), son n^o d'étage, sa surface en mètres carrés, et son nombre actuel de locataires occupants (trois entiers positifs ou nuls).
- fournir des fonctionnalités de base pour analyser une collection d'appartements.

Définition de classe objet. Définir une classe **Apartment** qui représente un appartement comme indiqué ci-dessus. Tous les attributs seront **private**. La classe doit offrir :

- un *constructeur vide*, qui ne fait que créer un nouvel appartement. Tous ses attributs auront la valeur par défaut du type de données correspondant.
- un *constructeur trivial*, qui reçoit en paramètres les valeurs à donner à tous les attributs d'un nouvel appartement ;
- tous les accesseurs, permettant de récupérer les valeurs de chacun des attributs ;
- et une méthode d'instance **toString()** qui rend en résultat une chaîne de caractères décrivant, en français, les attributs de l'appartement ;

Définition d'instances de classe. Dans une classe **ApartmentTest**, définir plusieurs exemples d'appartements au moyen de la classe **Apartment**. Puis, définir un exemple de parc immobilier locatif, en le modélisant comme un tableau d'appartements.

Utilisation de classe objet. Dans la classe **ApartmentTest**, définir des fonctions statiques permettant des analyses de base d'un parc immobilier locatif :

- la fonction **afficheParc** permet d'afficher tous les éléments d'un tableau d'appartements.
- la fonction **totalLocataires** permet, étant donné un tableau d'appartements, de calculer le nombre total de locataires occupants.
- la fonction **apptSurfaceMax** permet de trouver l'appartement qui a la plus grande superficie dans un tableau d'appartements. *Indication : Inspirez-vous de l'Exercice 7.1.*
- la fonction **surfaceMoyenne** permet de calculer la superficie moyenne d'un parc immobilier locatif. *Indication : Inspirez-vous de l'Exercice 7.3.*

Testez ces fonctions pour vous assurez de leur correction.